Week 3 - Class 1

From Prompt Engineering to Context Engineering - RAG Foundations

Today we solve a major weakness of plain LLM systems: missing context.

We will understand why private/company knowledge is not automatically known to the model, and how retrieval changes the system.

By the end of class, you should be able to explain RAG, chunking, retrieval, and why grounded context makes AI more useful.

This is where AI stops being just a model call and starts becoming a knowledge-connected system.

The Core Problem

A plain model can be powerful, but still fail on real company or domain-specific tasks.

- No access to private company data or internal docs
- Can forget earlier context in long conversations
- May hallucinate confidently when not grounded
- Can sound convincing without being correct

P.S: AI is often limited less by intelligence and more by missing context.

We did not make the model smarter.
We made the system smarter.

What is RAG?

RAG stands for Retrieval-Augmented Generation.

In simple language: first retrieve the right knowledge, then generate the answer.

The model is not magically remembering your company data. The system is feeding the right memory at the right time.

RAG Pipeline

This is the full system mental model students should leave with.

Prompt Engineering vs Context Engineering

Prompt Engineering

Controls instructions, behavior, format, and output style.

Context Engineering

Controls what knowledge the model sees before it answers.

Keyword Search vs Meaning Search

Keyword search looks for exact wording. It can miss related meaning.

Meaning-based retrieval tries to find semantically related text, even if wording differs.

Example: 'refund policy' and 'return rules' can be close in meaning, even if the words are different.

Next class we will replace our simplified retrieval logic with embedding-based similarity.

Why Chunking Matters

We do not retrieve full books or full PDFs every time.

Large chunks increase noise, token cost, and confusion.

Tiny chunks can lose meaning and context.

Good chunking is a precision trade-off: enough context, not too much.

P.S: Too much context can be almost as bad as no context.

Code Demo - Simplified Knowledge Base

We start with a fake company knowledge base before we move to embeddings in the next class.

```
knowledge_base = {
    "refund": "Refunds are allowed within 30 days with receipt.",
    "password": "Password resets require manager approval.",
    "timesheet": "Timesheets must be submitted every Friday."
}
```

Code Demo - Retrieval Function

This is simplified retrieval logic. Real RAG replaces this with similarity search.

```
def retrieve_context(question):
    q = question.lower()

    if "refund" in q or "return" in q:
        return knowledge_base["refund"]
    elif "password" in q:
        return knowledge_base["password"]
    elif "timesheet" in q:
        return knowledge_base["timesheet"]
    else:
        return "No relevant company policy found."
```

Code Demo - Minimal RAG System

This is the first real system upgrade to our AI Assistant: retrieve first, then answer.

```
def rag_system(question):
    context = retrieve_context(question)

    prompt = f"""
Answer the question using ONLY the context below.

Context:
{context}

Question:
{question}
"""

    return run_prompt(prompt)

print(rag_system("What is the refund policy?"))
```

Before vs After

LLM Only

User -> Model -> Guess

No private knowledge
Higher hallucination risk

RAG System

User -> Retrieval -> Context -> Model

Grounded answer
More useful in real work

Limitations of RAG

RAG reduces hallucination risk. It does not eliminate hallucination.

If retrieval is wrong, noisy, or incomplete, answers can still be weak.

Conflicting chunks can confuse the model.

Good AI systems still need validation, evaluation, and monitoring.

Real-World Uses

Company knowledge assistant

Customer support AI

HR and policy bot

Research assistant

Document question-answering system

Most useful AI products are not just models. They are models connected to knowledge.

A model without context is powerful but unreliable.
RAG gives AI the right memory at the right time.

Where We Are Now

- Week 1 → Understanding AI (probabilistic systems)
- Week 2 → Controlling AI (prompts, JSON, guardrails)
- Week 3 → Giving AI knowledge (retrieval, context, RAG)

What Was Missing

- Week 2 improved structure and control
- But the model still does NOT know company data
- Better formatting ≠ better truth

Why Week 2 Is Not Enough

- Even with prompts, temperature, JSON, guardrails
- The model can still hallucinate
- We controlled HOW it answers, not WHAT it knows

Do We Even Need AI?

- Without AI → rigid if/else rules
- With AI + context → interpretation and reasoning
- Database stores facts, LLM interprets meaning

RAG (Deeper View)

- RAG = Retrieval-Augmented Generation
- System design pattern (not model feature)
- Retrieve → Inject → Generate

System Comparison

- Before: User → LLM → Guess
- After: User → Retrieval → Context → LLM → Grounded Answer
- We improved the INPUT

Chunking Explained

- Split documents into meaningful pieces
- Goal: retrieve relevant context only
- Not too big, not too small

Chunking Tradeoffs

- Too large → noisy, expensive
- Too small → loses meaning
- Good chunking = balance

Full AI System (Week 2 + 3)

- User → Retrieval → Context → Prompt → LLM → JSON → Validation
- Combines control + knowledge

Evolution of AI Systems

- Week 1 → Raw model
- Week 2 → Controlled output
- Week 3 → Knowledge-aware system
- Now usable in real work

Top-K Retrieval

- User Question → Search → Rank → Top-K → LLM

- Example:
- Refund + damaged items

- Top-K Selected:
- - Refund policy
- - Damage policy

- We retrieve context, not answers

Full RAG System Architecture (Advanced)

- User → Query
- ↓
- Retriever (Search / Embeddings)
- ↓
- Top-K Chunks
- ↓
- Prompt Builder (Context + Instructions)
- ↓
- LLM (Reasoning)
- ↓
- Structured Output (JSON)
- ↓
- Validation Layer
- ↓
- Application Output

- RAG = System Design, not model feature